---------- CASE PROJECT: Media & Technology ----------

IMPORTING LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tabulate import tabulate
from datetime import datetime
```

UPLOADING DATASET

```
from google.colab import files
uploaded = files.upload()
```

```
df = pd.read_csv('Global YouTube Statistics.csv', encoding='latin1')
```

QUESTION 1

```
top_10 = df.sort_values(by='subscribers', ascending=False).head(10)

plt.figure(figsize=(12, 6))
bars = plt.bar(top_10['Youtuber'], top_10['subscribers'], color='lightgreen',edgecolor = 'black',linewidth = 1.2)
plt.xlabel('YouTuber Channel')
plt.ylabel('Number of Subscribers')
plt.title('Top 10 YouTube Channels by Subscribers')
plt.xticks(rotation=45, ha='right')

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2.0, height, f'{int(height):,}',
             ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()
```

QUESTION 2

```
df = df.dropna(subset=['category'])

category_avg = df.groupby('category')['subscribers'].mean()
top_category = category_avg.sort_values(ascending=False)

top_category = top_category.reset_index(name='Average Uploads')
top_category = top_category.rename(columns={'category': 'Category Name'})

print("Category with highest average subscribers:")
print(tabulate(top_category.head(1), headers='keys', tablefmt='plain', showindex=False, numalign = 'left'))
```

QUESTION 3

```
df = df.dropna(subset=['category','uploads'])
category_avg = df.groupby('category')['uploads'].mean().reset_index()

plt.figure(figsize=(12, 6))
bars = plt.bar(category_avg['category'], category_avg['uploads'], color='skyblue',edgecolor = 'black',linewidth = 1.2)
plt.xlabel('Category')
plt.ylabel('Average Number of Uploads')
plt.title('Average Video Uploads by YouTube Channels in Each Category')
plt.xticks(rotation=45, ha='right')

for bar in bars:
    height = bar.get_height()
```

```
    plt.text(bar.get_x() + bar.get_width() / 2.0, height, f'{height:.1f}',
             ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()
```

QUESTION 4

```
df = df.dropna(subset=['Country'])
country_counts = df['Country'].value_counts()
country_counts = country_counts.reset_index(name='No. of YT Channels')
country_counts = country_counts.rename(columns={'Country': 'Country Name'})

print("Top 5 countires with highest number of youtube channels:")
print(tabulate(country_counts.head(5), headers='keys', tablefmt='plain', showindex=False, numalign = 'left'))


top5 = country_counts.head(5)

plt.figure(figsize=(10, 6))
bars = plt.bar(top5['Country Name'], top5['No. of YT Channels'], color='yellow', edgecolor='black', linewidth=1.2)
plt.xlabel('Country')
plt.ylabel('Number of YouTube Channels')
plt.title('Top 5 Countries with Highest Number of YouTube Channels')
plt.xticks(rotation=45, ha='right')

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2.0, height, f'{int(height)}',
             ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()
```

QUESTION 5

```
df = df.dropna(subset=['category', 'channel_type'])
distribution_type = df.groupby(['category', 'channel_type']).size().reset_index(name='Count')
categories = distribution_type['category'].unique()
for cat in categories:
    print(f"Category: {cat}")
    sub_df = distribution_type[distribution_type['category'] == cat]
    for _, row in sub_df.iterrows():
        print(f"  {row['channel_type']}: {row['Count']}")
    print()


distribution_type = df.groupby(['category', 'channel_type']).size().unstack(fill_value=0)

distribution_type.plot(kind='bar', stacked=True, figsize=(14, 7), colormap='tab20')

plt.xlabel('Category')
plt.ylabel('Number of Channels')
plt.title('Distribution of Channel Types Across Categories (Stacked)')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Channel Type')
plt.tight_layout()
plt.show()
```

QUESTION 6

```
df = df.dropna(subset=['subscribers', 'video views'])
correlation = df['subscribers'].corr(df['video views'])
print("Correlation between subscriber count and total video views:", correlation)


plt.figure(figsize=(10, 6))

plt.scatter(df['subscribers'], df['video views'], alpha=0.5, color='purple')
plt.xlabel('Number of Subscribers')
plt.ylabel('Total Video Views')
plt.title('Correlation between Subscribers and Total Video Views')
plt.yscale('log')
```

```
plt.tight_layout()
plt.show()
```

QUESTION 7

```
df = df.dropna(subset=['category', 'lowest_monthly_earnings', 'highest_monthly_earnings'])
category_earnings = df.groupby('category').agg({
    'lowest_monthly_earnings': 'mean','highest_monthly_earnings': 'mean'
}).reset_index()

category_earnings = category_earnings.sort_values(by='highest_monthly_earnings', ascending=False)

x = np.arange(len(category_earnings['category']))

plt.figure(figsize=(16, 7))
bar1 = plt.bar(x - 0.20, category_earnings['lowest_monthly_earnings'], 0.35 , label='Average Lowest Earnings', color='skyblu
bar2 = plt.bar(x + 0.20, category_earnings['highest_monthly_earnings'], 0.35, label='Average Highest Earnings', color='coral

plt.xlabel('Category')
plt.ylabel('Monthly Earnings (USD)')
plt.title('Average Monthly Earnings by Category')
plt.xticks(x, category_earnings['category'], rotation=90, ha='right')

for i in range(len(category_earnings)):
    plt.text(x[i] - 0.20, category_earnings['lowest_monthly_earnings'].iloc[i] + 100,
             f"${category_earnings['lowest_monthly_earnings'].iloc[i]:,.0f}",
             ha='center', va='bottom', fontsize=9)
    plt.text(x[i] + 0.15, category_earnings['highest_monthly_earnings'].iloc[i] + 100,
             f"${category_earnings['highest_monthly_earnings'].iloc[i]:,.0f}",
             ha='center', va='bottom', fontsize=9)

plt.legend()
plt.tight_layout()
plt.show()
```

QUESTION 8

```
df = df.dropna(subset=['subscribers_for_last_30_days'])
plt.figure(figsize=(8, 4))
plt.hist(df['subscribers_for_last_30_days'],bins = 100,color = 'skyblue',edgecolor='black')
plt.xlabel('Subscribers Gained (last 30 days)')
plt.ylabel('Number of Channels')
plt.title('Distribution of Subscribers Gained')
plt.xscale('log')
plt.show()
```

QUESTION 9

```
df = df.dropna(subset=['lowest_yearly_earnings', 'highest_yearly_earnings'])

def outliers(data):
    Q1 = data.quantile(0.25)
    Q3 = data.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return data[(data < lower_bound) | (data > upper_bound)]

outliers_low = outliers(df['lowest_yearly_earnings'])
outliers_high = outliers(df['highest_yearly_earnings'])

print(f"Number of outliers in Lowest Yearly Earnings: {len(outliers_low)}")
print(f"Number of outliers in Highest Yearly Earnings: {len(outliers_high)}")


plt.figure(figsize=(10, 6))
plt.boxplot(
    [df['lowest_yearly_earnings'], df['highest_yearly_earnings']], labels=['Lowest Yearly Earnings', 'Highest Yearly Earning
    patch_artist=True
)
plt.ylabel('Yearly Earnings (USD)')
plt.title('Boxplot of Yearly Earnings from YouTube Channels')
plt.yscale('log')
```

```
plt.grid(True)
plt.show()
```

QUESTION 10

```
df = df.dropna(subset=['created_year'])

year_counts = df['created_year'].value_counts().sort_index()

plt.figure(figsize=(10, 6))
bars = plt.bar(year_counts.index.astype(str), year_counts.values, color='orange', edgecolor='black')
plt.xlabel("Year")
plt.ylabel("Number of Channels Created")
plt.title("Distribution of YouTube Channel Creation Over Years")
plt.xticks(rotation=45)
plt.grid(axis='y')

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, height, f'{int(height)}', ha='center', va='bottom')

plt.show()
```

QUESTION 11

```
df = df.dropna(subset=['Country', 'Gross tertiary education enrollment (%)'])

country_channel_counts = df['Country'].value_counts().reset_index()
country_channel_counts.columns = ['Country', 'num_channels']

channel_education_df = df[['Country', 'Gross tertiary education enrollment (%)']].drop_duplicates()
merged_df = pd.merge(country_channel_counts, channel_education_df, on='Country')

plt.figure(figsize=(10, 6))
plt.scatter(merged_df['Gross tertiary education enrollment (%)'], merged_df['num_channels'], color='teal')
plt.xlabel('Gross Tertiary Enrollment (%)')
plt.ylabel('Number of YouTube Channels')
plt.title('Relationship between Tertiary Enrollment and YouTube Channel Count')
plt.grid(True)
plt.show()
```

QUESTION 12

```
df = df.dropna(subset=['Country', 'Unemployment rate'])

country_channel_counts = df['Country'].value_counts().reset_index()
country_channel_counts.columns = ['Country', 'num_channels']

top10_countries = country_channel_counts.head(10)
unemployment_df = df[['Country', 'Unemployment rate']].drop_duplicates()

merged_df = pd.merge(top10_countries, unemployment_df, on='Country')
merged_df = merged_df.sort_values(by='Unemployment rate', ascending=False)

plt.figure(figsize=(12, 6))
bars = plt.bar(merged_df['Country'], merged_df['Unemployment rate'], color='lightblue', edgecolor='black')

for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 0.05,
        f'{height:.2f}',
        ha='center',
        va='bottom',
        fontsize=10,
    )

plt.xlabel('Country')
plt.ylabel('Unemployment Rate (%)')
plt.title('Unemployment Rate in Top 10 Countries by YouTube Channel Count')
plt.xticks(rotation=45)
plt.tight_layout()
```

```
plt.show()
```

QUESTION 13

```
df = df.dropna(subset=['Country', 'Urban_population', 'Population'])
df['urban_population_percentage'] = (df['Urban_population'] / df['Population']) * 100
country_urban_df = df.groupby('Country')['urban_population_percentage'].mean().reset_index()
average_urban_pop = country_urban_df['urban_population_percentage'].mean()
print(f"Average urban population percentage across countries with YouTube channels: {average_urban_pop:.2f}%")
```

QUESTION 14

```
ne = df[(df['Latitude'] >= 0) & (df['Longitude'] >= 0)].shape[0]
nw = df[(df['Latitude'] >= 0) & (df['Longitude'] < 0)].shape[0]
se = df[(df['Latitude'] < 0) & (df['Longitude'] >= 0)].shape[0]
sw = df[(df['Latitude'] < 0) & (df['Longitude'] < 0)].shape[0]

print(f"Channels in North-East quadrant: {ne}")
print(f"Channels in North-West quadrant: {nw}")
print(f"Channels in South-East quadrant: {se}")
print(f"Channels in South-West quadrant: {sw}")


df = df.dropna(subset=['Latitude', 'Longitude'])

plt.figure(figsize=(12, 6))
plt.scatter(df['Longitude'], df['Latitude'], alpha=0.5, s=15)
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.title('Distribution of YouTube Channels by Coordinates')
plt.grid(True)
plt.show()
```

QUESTION 15

```
df = df.dropna(subset=['Country', 'subscribers', 'Population'])
subs_pop_df = df.groupby('Country').agg({
    'subscribers': 'sum',
    'Population': 'first'
}).reset_index()
correlation = subs_pop_df['subscribers'].corr(subs_pop_df['Population'])
print(f"Correlation between total subscribers and country population: {correlation:.2f}")


plt.figure(figsize=(10, 6))
plt.scatter(subs_pop_df['Population'], subs_pop_df['subscribers'], alpha=0.6, color='cyan',edgecolors='black')

plt.xlabel('Country Population')
plt.ylabel('Total Subscribers')
plt.title('Correlation between Total Subscribers and Country Population')

plt.xscale('log')
plt.yscale('log')

plt.tight_layout()
plt.show()
```

QUESTION 16

```
df = df.dropna(subset=['Country', 'Population'])
country_channel_counts = df['Country'].value_counts().reset_index()
country_channel_counts.columns = ['Country', 'Channel_Count']
country_population = df.groupby('Country')['Population'].first().reset_index()
country_merged = pd.merge(country_channel_counts, country_population, on='Country')
top10 = country_merged.sort_values(by='Channel_Count', ascending=False).head(10)

plt.figure(figsize=(12, 7))
bars = plt.bar(top10['Country'], top10['Population'], color='orange', edgecolor='black')

for bar in bars:
```

```
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 0.5*1e7,
        f'{int(height):,}',
        ha='center',
        va='bottom',
        fontsize=10,
    )

plt.xlabel('Country')
plt.ylabel('Population')
plt.title('Population of Top 10 Countries with Most YouTube Channels')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

QUESTION 17

```
df = df.dropna(subset=['Country', 'subscribers_for_last_30_days', 'Unemployment rate'])
subs_unemploy_df = df.groupby('Country').agg({
    'subscribers_for_last_30_days': 'sum',
    'Unemployment rate': 'first'
}).reset_index()
correlation = subs_unemploy_df['subscribers_for_last_30_days'].corr(subs_unemploy_df['Unemployment rate'])
print(f"Correlation between subscribers gained in last 30 days and unemployment rate: {correlation:.2f}")
```

```
plt.figure(figsize=(10, 6))
plt.scatter(subs_unemploy_df['Unemployment rate'], subs_unemploy_df['subscribers_for_last_30_days'],
            alpha=0.6, color='blue')
plt.xlabel('Unemployment Rate (%)')
plt.ylabel('Subscribers Gained in Last 30 Days')
plt.title('Correlation between Subscribers Gained and Unemployment Rate of Country')
plt.yscale('log')
plt.tight_layout()
plt.show()
```

QUESTION 18

```
df = df.dropna(subset=['channel_type', 'video_views_for_the_last_30_days'])

plt.figure(figsize=(12, 6))
sns.boxplot(data=df, x='channel_type', y='video_views_for_the_last_30_days')
plt.xlabel('Channel Type')
plt.ylabel('Video Views in Last 30 Days')
plt.title('Distribution of Video Views in Last 30 Days Across Channel Types')
plt.xticks(rotation=45, ha='right')
plt.yscale('log')
plt.tight_layout()
plt.show()
```

QUESTION 19

```
df = df.dropna(subset=['created_month'])

season_map = {
    'Jan': 'Winter', 'Feb': 'Winter', 'Mar': 'Spring',
    'Apr': 'Spring', 'May': 'Spring', 'Jun': 'Summer',
    'Jul': 'Summer', 'Aug': 'Summer', 'Sep': 'Autumn',
    'Oct': 'Autumn', 'Nov': 'Autumn', 'Dec': 'Winter'
}

df['season'] = df['created_month'].map(season_map)

season_counts = df['season'].value_counts()

season_order = ['Winter', 'Spring', 'Summer', 'Autumn']
season_counts = df['season'].value_counts().reindex(season_order)

plt.figure(figsize=(8, 6))
bars = plt.bar(season_counts.index, season_counts.values, color='lightblue',edgecolor='black')
plt.ylabel("Number of Channels")
plt.title("YouTube Channels by Season")
```

```
plt.grid(axis='y')

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, height, f'{int(height)}', ha='center', va='bottom')

plt.show()
```

QUESTION 20

```
df = df.dropna(subset=['created_year', 'created_month', 'created_date', 'subscribers'])

df['created_year'] = df['created_year'].astype(int)

month_map = {
    'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4, 'May': 5, 'Jun': 6,'Jul': 7, 'Aug': 8, 'Sep': 9, 'Oct': 10, 'Nov': 11, 'Dec': 12
}

df['created_month_num'] = df['created_month'].map(month_map)

df['created_date'] = df['created_date'].astype(int)

df['created_full_date'] = pd.to_datetime(dict(
    year=df['created_year'],
    month=df['created_month_num'],
    day=df['created_date']
), errors='coerce')

df = df.dropna(subset=['created_full_date'])

now = pd.to_datetime(datetime.now())

df['months_since_creation'] = ((now - df['created_full_date']) / pd.Timedelta(days=30)).astype(int)
df = df[df['months_since_creation'] > 0]

df['subs_per_month'] = df['subscribers'] / df['months_since_creation']

overall_avg = df['subs_per_month'].mean()

print(f"Average subscribers gained per month since creation (overall): {overall_avg:.2f}\n")

plt.figure(figsize=(10, 6))
plt.hist(df['subs_per_month'], bins=50, color='purple', edgecolor='black', log=True)
plt.xlabel("Subscribers per Month")
plt.ylabel("Number of Channels (log scale)")
plt.title("Distribution of Subscribers Gained per Month")
plt.grid(axis='y')
plt.show()
```